

Course Outline: Debugging with Python

Title: Debugging with Python: Mastering Error Handling and Problem-Solving Techniques **Learnpack:** + Debugging with Python
Prerequisite: Error handling basics (try/catch concepts) **Target Audience:** Beginner Python developers learning to diagnose, handle, and resolve code issues **Total Lessons:** 8 **Format:** Mixed (50% hands-on)

Course Description

This course teaches students how to effectively debug Python applications using systematic approaches and built-in tools. Students will learn to read and interpret Python error messages, implement robust exception handling, and use debugging techniques to identify and resolve issues. By the end of this course, students will confidently diagnose problems in their code, handle errors gracefully, and communicate issues effectively to users.

The course emphasizes practical debugging skills that every Python developer needs: from understanding what went wrong to implementing solutions that prevent applications from crashing. Students will work with real debugging scenarios and learn to use Python's debugging tools effectively.

Course Philosophy

- This course emphasizes:
- **Evidence over assumption:** Using concrete debugging tools rather than guessing
 - **Graceful degradation:** Applications should handle errors without crashing
 - **User-focused error communication:** Translating technical errors into actionable user messages
 - **Systematic problem-solving:** Following consistent debugging methodologies

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Understanding Errors and Exceptions	2
02 - Exception Handling Fundamentals	2
03 - Debugging Techniques and Tools	2
04 - Assessment and Application	2
05 - Outro	1
Total	8

Section 00: Welcome

00.0 Welcome to Python Debugging 📖

Learning Objectives:

- Understand why debugging skills are essential for Python developers
- Identify the core problem this course solves: preventing application crashes

- Recognize the difference between reactive debugging and proactive error handling

Content Outline:

- The reality of bugs in software development
- Why applications crash and how debugging prevents it
- Preview of debugging tools and techniques you'll master
- Course journey: from error confusion to confident problem-solving

Transitions: Next, we'll explore the different types of errors Python can encounter and learn to distinguish between them.

Key Principles:

- Every developer encounters bugs - debugging skills separate beginners from professionals
- Systematic approaches beat random trial-and-error
- Good error handling improves user experience dramatically

Examples:

- A web scraper that crashes when a website is down vs. one that handles the error gracefully
 - An e-commerce app that shows "Error 500" vs. "We're having trouble processing your order. Please try again."
-

Section 01: Understanding Errors and Exceptions

01.0 Types of Python Errors and When They Occur

Learning Objectives:

- Distinguish between syntax errors, runtime errors, and logical errors
- Identify common Python exceptions and their typical causes
- Recognize error patterns that indicate specific types of problems

Content Outline:

- Syntax errors: caught before code runs (SyntaxError, IndentationError)
- Runtime exceptions: occur during execution (ValueError, TypeError, KeyError, FileNotFoundError)
- Logic errors: code runs but produces wrong results
- When each type typically occurs in the development cycle
- Common scenarios that trigger each exception type

Transitions: Now that you understand the types of errors Python encounters, let's learn how to read the traceback information Python provides when errors occur.

Key Principles:

- Different error types require different debugging strategies
- Error timing (compile vs. runtime) affects how you approach fixing them
- Understanding error categories helps you prevent them proactively

Examples:

- Syntax error: missing colon after if statement
- Runtime error: trying to access a dictionary key that doesn't exist
- Logic error: calculating total price but forgetting to include tax

Anti-patterns (woven in):

- **Ignoring error categories:** Treating all errors the same way instead of recognizing that syntax errors need different approaches than runtime exceptions
- **Panic debugging:** Seeing any error and immediately changing random code instead of reading what the error actually says

01.1 Reading and Interpreting Python Tracebacks

Challenge Prompt: Create a function that deliberately triggers different types of exceptions and practice reading the traceback information to identify the exact line and cause of each error.

Solution Code:

```
def debug_practice():
    # This will cause a NameError
    try:
        print(undefined_variable)
    except NameError as e:
        print(f"NameError caught: {e}")
        print(f"Error type: {type(e).__name__}")

    # This will cause a KeyError
    try:
        my_dict = {"name": "John"}
        print(my_dict["age"])
    except KeyError as e:
        print(f"KeyError caught: {e}")
        print(f"Error type: {type(e).__name__}")

    # This will cause a ValueError
    try:
        number = int("not_a_number")
    except ValueError as e:
        print(f"ValueError caught: {e}")
        print(f"Error type: {type(e).__name__}")

if __name__ == "__main__":
    debug_practice()
```

Challenge Code:

```
def debug_practice():
    # TODO: Create a try/except block that catches a NameError
    # Try to print a variable that doesn't exist

    # TODO: Create a try/except block that catches a KeyError
    # Try to access a dictionary key that doesn't exist

    # TODO: Create a try/except block that catches a ValueError
    # Try to convert an invalid string to an integer

    pass

if __name__ == "__main__":
    debug_practice()
```

Section 02: Exception Handling Fundamentals

02.0 Try/Catch/Finally Pattern in Python

Learning Objectives:

- Implement the try/except/finally pattern correctly in Python
- Understand when to use specific exception handling vs. general exception catching
- Design exception handling that maintains application stability

Content Outline:

- Basic try/except syntax and flow control
- Catching specific exceptions vs. general Exception class
- The finally block: guaranteed cleanup code
- Multiple except blocks for different error types
- Best practices for exception handling scope

Transitions: With the fundamentals of try/except/finally understood, let's practice building robust exception handlers that handle real-world scenarios gracefully.

Key Principles:

- **Scope Specific:** Use try/except specifically where dangerous operations occur, not around entire functions
- **Finally for cleanup:** Use finally blocks to ensure resources are cleaned up regardless of success or failure
- **Specific over general:** Catch specific exceptions when possible rather than broad Exception catching
- **Fail gracefully:** Applications should handle errors without crashing

Examples:

- File operations with proper cleanup using finally
- API calls with specific handling for network vs. data errors
- Database operations with connection cleanup

Anti-patterns (woven in):

- **Wrapping Gigante:** Putting entire applications or large functions in one massive try/catch instead of targeting specific risky operations
- **Silent failures:** Catching exceptions but not logging them or informing the user anything went wrong
- **Exception as control flow:** Using try/catch for normal program logic instead of just error handling

02.1 Building Robust Exception Handlers

Challenge Prompt: Build a file processing function that handles multiple types of errors gracefully and always cleans up resources, demonstrating proper exception handling patterns.

Solution Code:

```
def process_user_file(filename):
    file_handle = None
    try:
        print(f"Opening file: {filename}")
        file_handle = open(filename, 'r')

        content = file_handle.read()

        # Process the content (might raise ValueError)
        lines = content.split('\n')
        numbers = [int(line.strip()) for line in lines if line.strip()]

        total = sum(numbers)
        print(f"Successfully processed {len(numbers)} numbers. Total: {total}")
        return total

    except FileNotFoundError:
```

```

        print(f"Error: Could not find file '{filename}'. Please check the file path.")
        return None
    except ValueError as e:
        print(f"Error: File contains invalid data. {e}")
        return None
    except PermissionError:
        print(f"Error: Permission denied accessing '{filename}'.")
        return None
    finally:
        if file_handle:
            file_handle.close()
            print("File closed successfully.")

# Test the function
if __name__ == "__main__":
    result = process_user_file("numbers.txt")

```

Challenge Code:

```

def process_user_file(filename):
    file_handle = None
    try:
        # TODO: Open the file for reading
        # TODO: Read the content and split into lines
        # TODO: Convert each line to an integer (this might raise ValueError)
        # TODO: Calculate and return the sum
        pass
    except FileNotFoundError:
        # TODO: Handle file not found error
        pass
    except ValueError as e:
        # TODO: Handle invalid data in file
        pass
    except PermissionError:
        # TODO: Handle permission denied error
        pass
    finally:
        # TODO: Always close the file if it was opened
        pass

# Test the function
if __name__ == "__main__":
    result = process_user_file("numbers.txt")

```

Section 03: Debugging Techniques and Tools

03.0 Print Debugging vs Python Debugger (pdb)

Learning Objectives:

- Compare print debugging with systematic debugging tools
- Understand when to use print statements vs. interactive debugging
- Learn the basic commands and workflow of Python's pdb debugger

Content Outline:

- Print debugging: quick insights but limited scope

- Python debugger (pdb): interactive investigation of program state
- Setting breakpoints with `pdb.set_trace()`
- Basic pdb commands: step, next, continue, print variables
- When each approach is most effective

Transitions: Now let's put these debugging techniques into practice with an interactive debugging session where you'll solve a real bug using pdb.

Key Principles:

- **Evidence over assumption:** Use debugging tools to see actual program state rather than guessing what's happening
- **Systematic investigation:** Follow a consistent process to isolate problems
- **Right tool for the job:** Print debugging for quick checks, pdb for complex issues
- **Interactive exploration:** Debugging tools let you examine variables and execution flow step by step

Examples:

- Using print statements to track variable values through a calculation
- Using pdb to step through a loop that's producing unexpected results
- Setting breakpoints to examine the state when an exception occurs

Patterns (woven in):

- **Three-state debugging:** Identify where the bug occurs (before/during/after a specific operation), then focus investigation on that area
- **Binary search debugging:** Add debugging at the middle of a problematic section, then narrow down based on whether the issue is before or after that point

03.1 Interactive Debugging Session

Challenge Prompt: Debug a function that calculates student grades but produces incorrect results, using Python's pdb debugger to identify and fix the logical error.

Solution Code:

```
import pdb

def calculate_final_grades(students):
    final_grades = {}

    for student_name, scores in students.items():
        # Bug was here: didn't exclude the lowest score
        # Original: total = sum(scores)

        # Fixed: Remove lowest score before calculating average
        if len(scores) > 1:
            working_scores = scores.copy()
            working_scores.remove(min(working_scores))
        else:
            working_scores = scores

        total = sum(working_scores)
        average = total / len(working_scores)

        # Assign letter grade
        if average >= 90:
            letter_grade = 'A'
        elif average >= 80:
            letter_grade = 'B'
```

```

        elif average >= 70:
            letter_grade = 'C'
        else:
            letter_grade = 'F'

        final_grades[student_name] = {
            'average': round(average, 2),
            'letter': letter_grade
        }

    return final_grades

# Test data
students = {
    "Alice": [85, 92, 78, 96],
    "Bob": [72, 84, 69, 77]
}

if __name__ == "__main__":
    grades = calculate_final_grades(students)
    print(grades)

```

Challenge Code:

```

import pdb

def calculate_final_grades(students):
    # This function should drop each student's lowest score
    # before calculating their final grade average

    final_grades = {}

    for student_name, scores in students.items():
        # pdb.set_trace() # Uncomment this line to start debugging

        # BUG: This includes ALL scores instead of dropping the lowest
        total = sum(scores)
        average = total / len(scores)

        # Assign letter grade
        if average >= 90:
            letter_grade = 'A'
        elif average >= 80:
            letter_grade = 'B'
        elif average >= 70:
            letter_grade = 'C'
        else:
            letter_grade = 'F'

        final_grades[student_name] = {
            'average': round(average, 2),
            'letter': letter_grade
        }

    return final_grades

# Test data - Alice should get an A (average of 91), Bob should get a B (average of 77.67)
students = {
    "Alice": [85, 92, 78, 96], # Drop 78, average = (85+92+96)/3 = 91

```

```

"Bob": [72, 84, 69, 77]      # Drop 69, average = (72+84+77)/3 = 77.67
}

if __name__ == "__main__":
    # TODO: Uncomment pdb.set_trace() above and use these pdb commands:
    # 'n' (next line), 's' (step into), 'p variable_name' (print variable)
    # 'c' (continue), 'q' (quit debugger)
    grades = calculate_final_grades(students)
    print(grades)

```

Section 04: Assessment and Application

04.0 Error Communication and User Experience

Learning Objectives:

- Design error messages that help users rather than confuse them
- Implement user-friendly error handling in applications
- Balance technical accuracy with user comprehension in error communication

Content Outline:

- Translating technical errors into user-friendly messages
- Providing actionable next steps in error messages
- UI patterns for error states: pending, fulfilled, rejected
- When to log technical details vs. show user-friendly messages
- Call-to-action buttons in error scenarios (retry, contact support, go back)

Transitions: Let's now assess your complete understanding of Python debugging through a comprehensive evaluation.

Key Principles:

- **Human translation:** Convert technical errors into language users understand
- **Actionable guidance:** Always provide users with clear next steps
- **Graceful degradation:** Show helpful error states instead of broken interfaces
- **Appropriate detail:** Log technical information for developers, show friendly messages to users

Examples:

- Instead of "KeyError: 'user_id'" show "We couldn't find your account information. Please try logging in again."
- Instead of "ConnectionError" show "We're having trouble connecting to our servers. Please check your internet connection and try again."
- Error pages with "Try Again" buttons instead of dead ends

Patterns (woven in):

- **UI of three states:** Pending (show spinner), Fulfilled (show data), Rejected (show helpful error message)
- **Call to Action (CTA):** Every error message includes a clear action the user can take
- **Finally pattern:** Use finally blocks to ensure loading spinners disappear whether requests succeed or fail

Anti-patterns (woven in):

- **Technical dumping:** Showing users raw error messages like "Error 500: Unexpected token < in JSON at position 0"
- **Dead end errors:** Error messages that just say "Error" without any guidance on what to do next
- **Exposed sensitive data:** Accidentally revealing internal system information in error messages during debugging

04.1 Python Debugging Mastery Assessment

Learning Objectives:

- Demonstrate understanding of Python error types and their causes
- Show proficiency in exception handling best practices
- Apply debugging strategies to identify and resolve code issues
- Design user-friendly error communication

Question Format: Multiple choice

Questions:

1. **What type of error occurs when you try to access a dictionary key that doesn't exist?**

- A) `SyntaxError`
- B) `KeyError` ✓
- C) `ValueError`
- D) `TypeError`

2. **Which debugging approach is most appropriate for investigating a complex logical error in a 50-line function?**

- A) Add print statements throughout the entire function
- B) Use `pdb` to set a breakpoint and step through the problematic section ✓
- C) Rewrite the entire function from scratch
- D) Use `try/catch` around the whole function

3. **What is the main purpose of a finally block in exception handling?**

- A) To catch additional exceptions that weren't caught earlier
- B) To execute cleanup code whether an exception occurred or not ✓
- C) To display error messages to the user
- D) To retry the operation that failed

4. **Which is an example of good user-facing error communication?**

- A) `FileNotFoundError: [Errno 2] No such file or directory: 'data.txt'`
- B) `"Error 404"`
- C) `"We couldn't find the file you're looking for. Please check the filename and try again."` ✓
- D) `"Exception in line 23"`

5. **When should you catch the general Exception class instead of specific exception types?**

- A) Always, to make sure you catch everything
- B) Never, always catch specific exceptions
- C) Only as a last resort when you need to prevent any unhandled exception ✓
- D) When you're too lazy to figure out what specific exceptions might occur

6. **What's the main problem with "wrapping gigante" (putting large code blocks in try/catch)?**

- A) It uses too much memory
- B) It makes the code run slower
- C) It makes it harder to identify exactly where errors occur and handle them appropriately ✓
- D) It prevents any exceptions from being caught

Evaluation Criteria:

- 6/6 correct: Excellent debugging mastery
- 4-5 correct: Good understanding, review specific exception types
- 2-3 correct: Basic grasp, practice more with `pdb` and exception handling
- 0-1 correct: Needs significant review of all debugging concepts

Score Interpretation: Students scoring 4+ demonstrate readiness to debug Python applications independently. Lower scores indicate need for additional practice with hands-on debugging exercises.

Section 05: Outro

05.0 Your Debugging Journey Forward

Content Outline:

- **Course recap:** You've mastered reading Python tracebacks, implementing robust exception handling, using pdb for interactive debugging, and communicating errors effectively to users
- **Connection to bigger picture:** These debugging skills are essential for every Python developer - from simple scripts to complex web applications, you'll use these techniques daily
- **Next steps and continued learning:**
 - Practice debugging in your own projects by intentionally using pdb when issues arise
 - Learn advanced debugging tools like IDE debuggers and logging frameworks
 - Explore performance debugging and profiling tools
- **Resources for further exploration:**
 - Python's official debugging documentation
 - Advanced debugging techniques for web frameworks
 - Logging best practices for production applications
- **Encouragement and celebration:** You now have the systematic approach and tools to tackle any Python bug with confidence. Remember: every expert debugger started where you are now - the difference is having a methodical approach and the right tools.

Note: This is conclusion only - no theory, exercises, or assessment.
